

V3 Implementation Action Plan

Data, Model & Training Workstream

Based on: V2 code audit · novelty gap analysis · ChatGPT suggestions · independent literature research

1. Synthesis: What I Know, What’s Missing, What’s Feasible

1.1 Your Current System (V2) — Code-Level Understanding

After reading both Python files in full, here is the precise picture of what V2 does and what it does not do. This is the foundation everything else builds on.

Component	Current V2 State	Implication for V3
	50K pairwise samples. Targets: $\log(c) = 3 \cdot \log(r_{\text{soft}}/r)$ — FULLY ANALYTICAL, no REBOUND needed. 60% sampled in transition zone $[0.5\epsilon, 50\epsilon]$. Inputs: $[\log(r_{\text{soft}}), \log(m_i), \log(m_j)]$	c is a known function of r only. Vector NN needs different targets. Mass sampling: $m \in [0.001, 2.0]$ M — broader than the test case (1, 0.01, 0.005).
Model architecture	$3 \rightarrow 32 \rightarrow 32 \rightarrow 32 \rightarrow 1$ (scalar output). SiLU activations. Input normalisation. Predicts $\log(c)$; $\exp()$ at inference.	Minimal change to add Vector NN: change output layer to 3 units, change targets to force vectors.
	3-tier gating: (1) $r > 500\epsilon$: exact Newtonian (2) $r \in [5e-4, 0.15 \text{ AU}]$: $c \cdot F_{\text{soft}}$ (3) $r < 5e-4$: pure softened baseline NN called for $\sim 1.1\%$ of steps.	Vector NN slots into tier (2) only. Tiers (1) and (3) unchanged. Total architecture change is small.
Adaptive sub-stepping	Triggers when $r_{\text{min}} < 0.05 \text{ AU}$. $n_{\text{sub}} = \min(16, \text{ceil}(0.05/r_{\text{min}}))$. Velocity-Verlet sub-steps.	Same for all models—no change needed. Can be toggled off ($\text{adapt_thresh}=0.0$) for before/after comparison.
	Single initial condition: $m=[1,0.01,0.005]$, fixed x_0, v_0 . 5 dt values: $[0.005, 0.01, 0.02, 0.04, 0.08]$. Metrics: RMS error, λ , speed-accuracy.	Need 2–3 more initial conditions. Need λ per model (trivial: reuse fit_log_slope). Need energy tracking (small addition).
What’s NOT in the code	No Vector NN. No energy tracking. No before/after adaptive comparison. No multi-IC evaluation. No NN invocation rate logged per-model.	All gaps, all addressable with modest code additions.

1.2 The Novelty Gap — Where the Paper Currently Stands

My research confirms the following: the literature on neural networks for N-body gravitational problems (Breen 2020, Ulibarrena 2024, Kipf 2018) **does not contain a systematic comparison of scalar magnitude vs full vector force prediction in terms of chaotic divergence rates**. This gap is real and is the strongest available novelty claim.

- **What exists in literature:** HNN vs DNN comparisons (Ulibarrena 2024); direct position prediction (Breen 2020); graph-NN for N-body (Kipf 2018). None isolate the scalar/direction decomposition.
- **What SIMON currently claims:** 'We are faster than ias15 and bodies stay bounded.' This is an engineering result, not a scientific discovery.
- **What SIMON could claim with one new experiment:** 'Constraining force direction to analytic geometry reduces error-growth rate in chaotic gravitational systems — quantified as a lower divergence exponent λ across 100-year rollouts.' This is a scientific principle.
- **The key question that unlocks this claim:** Does a Vector NN (which also learns direction) have a higher λ than SIMON (which fixes direction analytically)? Answering this is the purpose of Steps 1–3 below.

■ **Key research insight from literature scan:** Equivariant/invariant network design for molecular force fields (e.g., SchNet, DimeNet, NequIP) achieves stability precisely by enforcing rotational symmetry — meaning direction comes from geometry, not learned parameters. SIMON implements this principle for gravitational forces without naming it. Framing your contribution this way connects to a broader, active research thread.

1.3 Additional Ideas from Independent Research (Not in ChatGPT or SIMON Report)

Beyond what was already suggested, my research identified three additional feasible improvements:

Idea	What it is	Why novel/feasible	Priority
Rotation-invariance framing	Explicitly note that SIMON's scalar output is rotationally invariant by construction; the Vector NN is not	Connects to equivariant NN literature (NequIP, SchNet). No code change—writing only. Strong theoretical anchor for the claim.	HIGH Writing
Training seed sensitivity	Run training 3–5 times with different seeds; report mean $\pm$ std of val MSE and downstream λ	Reviewers always ask about result robustness. Adds 15 mins compute. Currently completely absent.	MEDIUM Code
Multi-IC evaluation	Test 3–5 different initial conditions beyond the fixed (x0,v0) in the code. Report mean+variance of all metrics.	The paper's single IC is its biggest generalizability weakness. Trivial to add—just define 2–4 more x0,v0 arrays.	MEDIUM Code

2. Step-by-Step Implementation Plan

All steps are ordered by dependency. Phase 1 is foundational (must come first). Phases 2–4 can overlap once Phase 1 is done. Each step states its objective, its connection to the research objective, how it fills the novelty gap, and the specific code changes required.

PHASE 1 — The Core New Experiment (Steps 1–3) These three steps constitute the single most important experimental addition. They must be done in sequence. Estimated total compute time: 30–60 minutes.

Step 1 Build and Train the Vector NN Baseline

Phase 1

Effort: ~2 hrs (code + training)

Produces: new file: train_vector_nn.py | new model: vector_nn.pt

Objective

Train a neural network that predicts the **full pairwise force vector (F_x, F_y, F_z) directly**, rather than a scalar correction factor. This serves as the baseline to test whether fixing direction analytically (SIMON's approach) matters for chaotic stability.

Connection to Research Objective

The paper's research question is: how closely can physically constrained NN force models approximate the long-term stability of traditional integrators? The Vector NN answers the counterfactual: *what happens without* the physical constraint of analytic direction? This makes 'physically constrained' a testable difference, not just a design choice.

How This Fills the Novelty Gap

No published paper compares scalar-magnitude vs full-vector force prediction for gravitational three-body chaotic stability. This experiment is the paper's unique contribution.

Exact Code Changes

Create a new file **train_vector_nn.py** (based on train_pair_correction_new.py):

```
# New class: same architecture, 3 outputs instead of 1
class VectorNN(nn.Module):
    def __init__(self, hidden=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(5, hidden), nn.SiLU(), # 5 inputs: [rx,ry,rz, log_mi, log_mj]
            nn.Linear(hidden, hidden), nn.SiLU(),
            nn.Linear(hidden, hidden), nn.SiLU(),
            nn.Linear(hidden, 3), # output: (Fx, Fy, Fz) correction
        )

# New training data generator: same sampling, different targets
def generate_vector_training_data(n=50000, eps=3e-4, seed=42):
    rng = np.random.RandomState(seed)
```

```

# Same r sampling as current (60% transition zone)
log_r = ... # same as current generate_training_data()
r = np.exp(log_r)
log_mi = rng.uniform(np.log(0.001), np.log(2.0), n)
log_mj = rng.uniform(np.log(0.001), np.log(2.0), n)
# Random unit direction vectors for each pair
dirs = rng.randn(n, 3); dirs /= np.linalg.norm(dirs, axis=1, keepdims=True)
r_soft = np.sqrt(r**2 + eps**2)
mi, mj = np.exp(log_mi), np.exp(log_mj)
# Analytic softened force magnitude
F_mag = mi * mj / r_soft**3 # G=1
# Full force vector (target)
F_vec = F_mag[:, None] * dirs # (n, 3)
features = np.stack([dirs[:,0]*r, dirs[:,1]*r, dirs[:,2]*r, log_mi, log_mj], axis=1)
return features.astype(np.float32), F_vec.astype(np.float32)

# Loss: MSE on force vectors (not log-space)
loss_fn = nn.MSELoss() # on (Fx, Fy, Fz) directly

```

■ Design note: The Vector NN receives the full relative position vector (rx, ry, rz) as input, not just the scalar distance. This is the key difference from SIMON. If it only received |r|, it couldn't predict direction. This means it has 5 inputs vs SIMON's 3 inputs. This is intentional and fair: the Vector NN uses strictly more information and should in principle perform at least as well on force prediction. The question is whether it performs better or worse on long-horizon chaotic stability.

Step 2 Integrate Vector NN into the Simulation Loop

Phase 1

Effort: ~3 hrs (code)

Produces: updated pair_eval.py with 3-model support

Objective

Add a second simulation mode to **pair_eval_after_adaptive.py** that uses the Vector NN to compute forces, while keeping the same integration loop, adaptive sub-stepping, and evaluation framework.

Connection to Research Objective

For the comparison to be scientifically valid, both models must run through *exactly the same* integration infrastructure. Same dt values, same initial conditions, same output format. Only the force computation changes.

Exact Code Changes

```

# Add to pair_eval_after_adaptive.py:

class VectorNN(nn.Module): ... # same definition as in train_vector_nn.py

def extract_weights_vector_numpy(model):
    # Same pattern as extract_weights_numpy() but for 5-input, 3-output network

```

```

sd = model.state_dict()
return {'w0T': sd['net.0.weight'].cpu().numpy().T.astype(np.float32).copy(), ...}

def compute_acc_vector(pos, w, Gmimj, ii, jj, inv_mi, inv_mj, log_mi, log_mj, nn_thresh, cfg):
rij = pos[jj] - pos[ii] # (P, 3) relative position vectors
r2 = np.einsum('ij,ij->i', rij, rij)
r = np.sqrt(r2 + 1e-30)
# Far field: exact Newtonian (same as SIMON tier 1)
invr3 = 1.0 / (r2 * r + 1e-30)
F_scalar = Gmimj * invr3
F_vec = F_scalar[:, None] * rij # (P, 3)
# Close encounter: replace with Vector NN prediction
close_mask = r < nn_thresh
if np.any(close_mask):
rij_close = rij[close_mask] # (nc, 3) - raw position vectors
nn_in = np.empty((nc, 5), dtype=np.float32)
nn_in[:, :3] = rij_close # direction information provided
nn_in[:, 3] = log_mi[close_mask]
nn_in[:, 4] = log_mj[close_mask]
# NN forward pass -> (Fx, Fy, Fz) directly
F_nn = vector_nn_forward(nn_in, w) # (nc, 3)
F_vec[close_mask] = F_nn # replace tier-2 forces
return accumulate_accelerations(F_vec, ii, jj, inv_mi, inv_mj)

# Main: add --model_type argument: 'hybrid' (current SIMON) or 'vector'
parser.add_argument('--model_type', choices=['hybrid','vector'], default='hybrid')

```

Step 3 Run the Three-Model Comparison + Divergence Rate Analysis

Phase 1

Effort: ~2 hrs (run + plot)

Produces: new figures: divergence_3model.png, speed_accuracy_3model.png

Objective

Run both SIMON and Vector NN through the full 100-year simulation at $dt=0.04$ (the reference dt). Compute divergence rate λ for each. Plot both on the same log-divergence-vs-time figure.

The Central Question Being Answered

Is $\lambda(\text{Vector NN}) > \lambda(\text{SIMON})$? If yes: you have proved that magnitude decomposition reduces error-growth rate in chaotic gravitational systems. This is the paper's core scientific claim. If λ values are similar: the contribution is still valid (SIMON achieves comparable accuracy with fewer learned degrees of freedom), but the claim must be reframed.

New Plot to Add

```
def plot_divergence_3model(times, delta_simon, delta_vector, slope_s, slope_v, window, op):
```

```
plt.figure(figsize=(8, 5))
plt.semilogy(times, delta_simon, 'b-', label=f'SIMON (Magnitude NN),  $\lambda$ ={{slope_s:.3f}}/yr')
plt.semilogy(times, delta_vector, 'r--', label=f'Vector NN,  $\lambda$ ={{slope_v:.3f}}/yr')
plt.axvspan(window[0], window[1], alpha=0.15, label='fit window')
plt.xlabel('Time (yr)'); plt.ylabel('RMS Position Error')
plt.title('Divergence Rate: Magnitude NN vs Vector NN')
plt.legend(); plt.grid(True, which='both', alpha=0.3)
plt.tight_layout(); plt.savefig(op, dpi=300); plt.close()

# Also add to the speed-accuracy frontier plot:
# Plot SIMON points in blue, Vector NN points in red
# Both on the same axes -> shows whether Vector NN trades speed for accuracy differently
```

■ **Expected result based on physics reasoning:** The Vector NN has to learn both magnitude and direction simultaneously. In a chaotic system, small errors in direction accumulate multiplicatively over time steps. SIMON eliminates directional learning error entirely by computing direction from exact geometry. We expect $\lambda(\text{Vector NN}) > \lambda(\text{SIMON})$, confirming that the physics constraint is doing real work. Even if the magnitudes differ only slightly, this is a clean, publishable result.

PHASE 2 — Robustness & Completeness (Steps 4–5) These steps strengthen the results without requiring the Vector NN to be done first, but benefit from it being done. Estimated compute: 1–2 hours.

Step 4 Add Multi-Initial-Condition Evaluation

Phase 2

Effort: ~2 hrs (define ICS + run)

Produces: updated perf_summary.txt | error bar figures

Objective

Run the full evaluation pipeline (currently hardcoded to one IC) on 3–5 distinct initial conditions spanning different dynamical regimes. Report mean $\pm$ std of all key metrics.

Connection to Research Objective

The paper claims results about the 'unrestricted three-body problem' but demonstrates them for a single configuration. Multi-IC evaluation is the minimal evidence needed to support generalization claims.

Three Additional Initial Conditions to Add

```
# IC Library to add to pair_eval_after_adaptive.py:
INITIAL_CONDITIONS = {
  'default': { # current IC
    'm': [1.0, 0.01, 0.005],
    'x0': [[0,0,0],[1,0,0],[0,1.2,0]],
    'v0': [[0,0,0],[0,1,0],[-0.9,0,0]]
  },
  'near_equal_mass': { # more chaotic regime
    'm': [1.0, 0.5, 0.25],
    'x0': [[0,0,0],[1,0,0],[-0.5,0.8,0]],
    'v0': [[0,0,0],[0,0.6,0],[-0.4,-0.3,0]]
  },
  'tight_binary': { # test close encounters more aggressively
    'm': [1.0, 0.01, 0.005],
    'x0': [[0,0,0],[0.3,0,0],[0,2.0,0]],
    'v0': [[0,0,0],[0,1.5,0],[-0.3,0,0]]
  },
  'hierarchical': { # restricted-like: one far body
    'm': [1.0, 0.01, 0.005],
    'x0': [[0,0,0],[1,0,0],[0,5.0,0]],
    'v0': [[0,0,0],[0,1,0],[-0.1,0,0]]
  },
}

# Loop over ICs, run both SIMON and Vector NN, collect metrics
# Report: mean_lambda, std_lambda, mean_RMS, std_RMS, frac_bounded
```

Step 5 Add Energy Drift Tracking

Phase 2

Effort: ~1 hr (code only)

Produces: new plot: energy_drift.png | updated perf_summary.txt

Objective

Track total mechanical energy $E(t) = KE + PE$ throughout each simulation run. Report energy drift $\Delta E(t)/E(0)$ as a function of time for each model.

Why This Matters

ias15 conserves energy to near-machine-precision. SIMON uses exact Newtonian gravity 98.9% of the time, so its energy should be nearly conserved. A Vector NN that learns direction may introduce systematic energy bias. This gives a *second independent metric* to compare models, complementing the position RMS and divergence rate.

Code Change (Small — Add to `compute_acc()` return value)

```
# Add energy computation to simulate_leapfrog_hybrid():
# At each recorded timestep, compute:
def compute_energy(pos, vel, m, G=1.0, eps=3e-4):
    N = len(m)
    KE = 0.5 * np.sum(m[:, None] * vel**2) # kinetic energy
    PE = 0.0
    for i in range(N):
        for j in range(i+1, N):
            r = np.sqrt(np.sum((pos[i]-pos[j])**2) + eps**2)
            PE -= G * m[i] * m[j] / r # potential energy
    return KE + PE

# Store E_out array alongside pos_out, vel_out
# Plot: (E(t) - E(0)) / |E(0)| vs time for ias15, SIMON, Vector NN
```


PHASE 3 — Before/After Adaptive + Training Robustness (Steps 6–7) These fill the parked gaps from the SIMON report and add training robustness. No dependency on Phases 1–2.

Step 6 Generate Before/After Adaptive Sub-Stepping Comparison

Phase 3

Effort: ~1 hr (run + table)

Produces: new table in paper: Gap G2 filled

Objective

Run SIMON at the reference dt with adaptive sub-stepping **disabled** (`adapt_thresh=0.0`) and re-run with it enabled. Record and tabulate: bodies bounded?, divergence slope, worst-case RMS error, per-step time.

Why This Is Critical

The SIMON report showed worst-case error drops from 143 to 1.54 (93×) with adaptive stepping. This number is currently absent from the paper. It is the single most dramatic result in the entire project and should be front-and-centre in Section 3.7 or a new Section 4.4.

Code Change (One Line)

```
# In pair_eval_after_adaptive.py main(), add a --no_adaptive flag:
parser.add_argument('--no_adaptive', action='store_true',
                    help='Disable adaptive sub-stepping for comparison')

# In simulate_leapfrog_hybrid(), change:
adapt_thresh = 0.0 if args.no_adaptive else 0.05 # 0.0 disables sub-stepping

# Run twice: with and without. Collect metrics. Build comparison table.
```

Step 7 Training Robustness: Multi-Seed Evaluation

Phase 3

Effort: ~1 hr (5x retraining)

Produces: mean $\pm$ std of val MSE and λ across seeds

Objective

Train the SIMON model 5 times with different random seeds (42, 123, 456, 789, 1000). For each trained model, record: val MSE, λ at the reference dt, final RMS at dt=0.04. Report mean $\pm$ std.

Why Reviewers Will Ask for This

The paper reports a single training run. If results are seed-sensitive, the claims may not be reproducible. If they are robust (expected, since the training target $c(r)$ is a smooth analytic function), that's an important positive result to state explicitly.

```
# In train_pair_correction_new.py, add seed loop:
seeds = [42, 123, 456, 789, 1000]
results = []
for seed in seeds:
    model, tl, vl = train_model(feats, targ, hidden=32, epochs=5000,
                                device=device, seed=seed)
```

```
results.append({'seed': seed, 'best_val_mse': min(vl)})  
torch.save(model.cpu().state_dict(), f'pair_correction_nn_seed{seed}.pt')  
# Then evaluate each saved model through pair_eval.py and collect lambda
```

PHASE 4 — Framing & Paper Updates (Steps 8–9) Pure writing. Do concurrently with Phases 1–3 or after results are in.

Step 8 Add Rotation-Invariance Framing to Section 3.4

Phase 4

Effort: ~1 hr (writing)

Produces: stronger theoretical motivation for SIMON's architecture

Objective

Add a paragraph to Section 3.4 (Physically Constrained Force Magnification Neural Network) explicitly stating that predicting a scalar correction factor makes SIMON's force model **rotationally invariant by construction**, and connecting this to the broader equivariant NN literature.

The Text to Add (~100 words)

"The scalar output of PFMN is rotationally invariant: the correction factor c depends only on inter-body distance r and masses, not on the orientation of the coordinate frame. The force direction is recovered analytically from the relative position vector, which is equivariant under rotation by construction. This is in contrast to vector force networks that predict (F_x, F_y, F_z) directly, which must either be designed with equivariant architectures (e.g., NequIP, SchNet) or learn rotational symmetry from data. SIMON achieves rotational equivariance for free through physics-based direction computation, eliminating an entire class of learning error."

Step 9 Write the Abstract + Update Research Question

Phase 4

Effort: ~2 hrs (writing)

Produces: abstract added; Sec 3.1 research question updated

Objective

Write the missing abstract and update Section 3.1's research question to explicitly reference the magnitude vs vector comparison.

Abstract Template (fill in V3 results when ready)

The Three-Body Problem is a canonical chaotic system for evaluating physically constrained neural network force models. We present SIMON, a hybrid pairwise force magnitude network that predicts a scalar correction to analytic softened gravity rather than a full force vector, enforcing rotational equivariance by construction. We compare SIMON against a Vector NN baseline that learns both force magnitude and direction, and against the reference ias15 integrator. Across 100-year simulations of an unrestricted three-body system, SIMON achieves a [X]% lower divergence rate than the Vector NN baseline, while completing simulations [2.09×] faster than ias15 with all bodies remaining gravitationally bounded. These results demonstrate that enforcing physics-based direction constraints reduces long-horizon error accumulation in chaotic gravitational systems, and that implementation efficiency — not algorithmic novelty — is the primary determinant of computational advantage.

3. Dependency Map & Recommended Sequence

Step	Title	Depends on	Blocks	Est. Time
Step 1	Train Vector NN	Nothing	Steps 2,3,7	2 hrs
Step 2	Integrate VectorNN into eval loop	Step 1	Step 3	3 hrs
Step 3	3-model comparison + λ	Steps 1,2	Paper Sec 4	2 hrs
Step 4	Multi-IC evaluation	Nothing (or Step 2)	Paper Sec 4	2 hrs
Step 5	Energy drift tracking	Nothing	Paper Sec 4	1 hr
Step 6	Before/after adaptive	Nothing	Paper Sec 3.7	1 hr
Step 7	Training seed robustness	Step 1 (for VectorNN)	Paper appendix	1 hr
Step 8	Rotation-invariance framing	Nothing	Paper Sec 3.4	1 hr
Step 9	Abstract + updated RQ	Steps 1-3 results	Paper Sec 1,3.1	2 hrs

■ **Recommended starting sequence:** Step 1 (Vector NN training — independent, foundational) → Step 6 (Before/After adaptive — one-line code change, fills a big gap immediately) → Step 4 (Multi-IC evaluation — can run while Step 1 trains) → Step 2 → Step 3 (complete the core comparison) → Steps 5, 7, 8, 9 (in any order).

3.1 What V3 Will Be Able to Claim (if Steps 1–3 confirm the hypothesis)

- **Main claim:** Constraining neural force models to predict scalar magnitudes with analytic direction reduces long-horizon divergence rates in chaotic gravitational systems compared to unconstrained vector prediction. [Supported by Step 3: λ comparison]
- **Supporting claim 1:** The hybrid correction architecture achieves 2.09× speedup over ias15 while maintaining bounded orbital dynamics across all tested initial conditions. [Supported by Steps 2,4]
- **Supporting claim 2:** Adaptive distance-based sub-stepping reduces worst-case RMS error by 93× and eliminates body ejections, at a cost of only 33% additional per-step overhead. [Supported by Step 6]
- **Supporting claim 3:** Scalar correction factors are rotationally invariant by construction — this provides a theoretical basis for the stability advantage without requiring equivariant architecture design. [Supported by Step 8]
- **Methodological claim:** Divergence-rate proxies (λ from error-growth curves) are more informative than final-state RMS error for evaluating ML force models in chaotic systems, as they characterise long-horizon error accumulation rather than point-in-time accuracy. [Supported by Step 3]

End of V3 Implementation Action Plan | 9 Steps | ~15 hours total | 5 require new experiments